ns-3 Installation and Examples

Xuanhao Luo

ns-3 Installation Guide：https://www.nsnam.org/docs/release/3.38/installation/singlehtml/index.html

Reference: https://www.nsnam.com/2022/11/installing-ns-337-and-ns-335-in-ubuntu.html

Introduction to ns-3

ns-3 is an open-source discrete-event network simulator used primarily for research and educational purposes. It enables the simulation of both *wired and wireless network protocols and systems,* making it a versatile tool for networking research.

Why Use ns-3?

- **Educational Purposes:** It's a great tool for learning about network protocols and systems.
- **Research and Development:** ns-3 helps in developing new network technologies and testing them in simulated environments.
- **Performance Testing:** It provides insights into how network changes might affect performance before implementing them in real-world scenarios.

Install Ubuntu （if you are using windows）

Download and Install VMware Workstation Player

Download Ubuntu 20.04 ISO

Create a New Virtual Machine

Installation Requirements

1. **C++ Compiler:** Ensure you have g++ version 9 or higher, or clang++.
2. **Python:** Version 3.6 or higher is required.
3. **CMake:** Version 3.10 or higher.
4. **Git:** To clone the ns-3 repository.
5. **Other Tools:** These include make, ninja-build, and several libraries and packages that ns-3 depends on.

Install Dependencies

```
sudo apt update
sudo apt install g++ python3 cmake ninja-build git
sudo apt install ccache
sudo apt install python3-pip
python3 -m pip install --user cppyy
sudo apt install gir1.2-goocanvas-2.0 python3-gi python3-gi-cairo python3-pygraphviz gir1.2-gtk-3.0 ipython3
sudo apt install python3-setuptools git
sudo apt install qtbase5-dev qtchooser qt5-qmake qtbase5-dev-tools
sudo apt install openmpi-bin openmpi-common openmpi-doc libopenmpi-dev
sudo apt install mercurial unzip
```

```
sudo apt install gdb valgrind
sudo apt install clang-format
sudo apt install doxygen graphviz imagemagick
sudo apt install texlive texlive-extra-utils texlive-latex-extra texlive-font-utils dvipng latexmk
sudo apt install python3-sphinx dia
sudo apt install gsl-bin libgsl-dev libgslcblas0
sudo apt install tcpdump
sudo apt install sqlite sqlite3 libsqlite3-dev
sudo apt install libxml2 libxml2-dev
sudo apt install libgtk-3-dev
sudo apt install vtun lxc uml-utilities
sudo apt install libxml2 libxml2-dev libboost-all-dev
```

RedHat: Install a package:
`sudo yum install <package_name>`

install all necessary packages in one go:

```
sudo apt update
sudo apt install build-essential autoconf automake libxmu-dev g++ python3 python3-dev pkg-config sqlite3 cmake
```

Download ns-3.36 from official website

- **Download ns-3.36:** Visit the official ns-3 website and download version 3.36. https://www.nsnam.org/releases/ns-3-36/
- Save the downloaded tarball in a new directory.
- **Create and Navigate to tarballs Directory:**

```
mkdir tarballs
cd tarballs
```

Extract the ns-3 tarball inside the “tarballs” directory.

```
tar -xvf ns-allinone-3.36.tar.bz2
```

Download ns-3 Using Git:

Next, we'll download the source code using Git from the ns-3 repository hosted on GitLab (https://gitlab.com/nsnam/ns-3-allinone).

This process involves forking or cloning the repository to your local environment. Run these commands:

```
cd
mkdir workspace
cd workspace
```

```
git clone https://gitlab.com/nsnam/ns-3-allinone.git
cd ns-3-allinone
```

Once inside the ns-3-allinone directory, download the desired version of ns-3. Here, we will download version 3.36.

```
python3 download.py -n ns-3.36
```

After downloading, your project directory structure will look like this:

```
workspace/
└── ns-3-allinone/
    ├── bake
    ├── ns-3-dev
    ├── ns-3.36
    ├── ...
```

Build

Navigate to the ns-3.36 Directory.

Configure the Build Use the ns3.py script to configure the build, enabling examples and tests.

```
cd ns-allinone-3.36/ns-3.36
./ns3 configure --enable-examples --enable-tests
```

Build ns-3 using the ns3.py script.

```
./ns3 build
```

./ns3 build

`./ns3 build` is a command used to compile the ns-3 project using the build system. This command involves several steps to transform the ns-3 source code into executable binary files and libraries. Here's a detailed explanation of the `./ns3 build` operation:

1. **Preprocessing**
   Before compiling the source code, the preprocessing step involves parsing configuration files, setting compiler options, and defining macros. The preprocessor handles all #include directives, macro definitions, and conditional compilation directives.
2. **Compilation**
   Each source file (usually .cc or .cpp files) is individually compiled by the compiler into **object files (typically .o files)**. The compiler converts the C++ code into machine code, but at this stage, these object files cannot run independently.
3. **Assembly**
   In some build systems, after compilation, there is an assembly step where the intermediate code generated by the compiler is *converted into machine instructions*. However, in modern compilers, compilation and assembly are often completed in a single step.

4. **Linking**
   The linker combines all the object files and library files into the final executable or library files. The linking step resolves symbols in the object files to specific memory addresses and integrates all code and data segments together.
5. **Packaging**
   Preparing the compiled code for distribution, which might include creating installers or compressed archives.

`./ns3` is a command used in the latest versions of the ns-3 network simulator to manage the build and execution process.

It replaces the older `waf` build system previously used by ns-3. The `./ns3` tool provides a streamlined way to configure, build, and run ns-3 simulations.

Run the Test Suite:

After building ns-3, it is crucial to verify that the installation is successful by running the test suite.

```
./test.py
```

If the tests pass without errors, it indicates that the installation is complete and all modules are functioning correctly.

Run the First Script:
To further confirm the setup, run the first example script provided by ns-3.

```
./ns3 run first
```

Get Help on the First Script:

You can also get help on running the first script with various options

```
./ns3 run 'first --PrintHelp'
```

Examples

first.cc

- This code sets up a basic network simulation using the ns-3 simulator. It creates two nodes, connects them with a *point-to-point link*, assigns IP addresses, and installs UDP echo server and client applications on the nodes.
- The server listens for packets and the client sends one packet to the server, demonstrating basic network communication in ns-3.

Network Topology:

```
//       10.1.1.0
// n0 -------------- n1
//    point-to-point
```

```
#include "ns3/core-module.h"
#include "ns3/network-module.h"
#include "ns3/internet-module.h"
#include "ns3/point-to-point-module.h"
#include "ns3/applications-module.h"


using namespace ns3;

// Define a log component
NS_LOG_COMPONENT_DEFINE ("FirstScriptExample");

int
main (int argc, char *argv[])
{
  CommandLine cmd (__FILE__);
  cmd.Parse (argc, argv);

  // Set the time resolution, which can only be set once.
  // Dynamic setting requires a lot of memory, so once set, it will free up memory.
  // If not set, it uses the default value of nanoseconds (ns).
  Time::SetResolution (Time::NS);

  // Enable two log components, which are built into the EchoClient and EchoServer applications.
  LogComponentEnable ("UdpEchoClientApplication", LOG_LEVEL_INFO);
  LogComponentEnable ("UdpEchoServerApplication", LOG_LEVEL_INFO);



  // Create node objects and store Node object pointers internally.
  NodeContainer nodes;
  nodes.Create (2);



  // Set network device and channel attributes.
  PointToPointHelper pointToPoint;
  pointToPoint.SetDeviceAttribute ("DataRate", StringValue ("5Mbps"));
  pointToPoint.SetChannelAttribute ("Delay", StringValue ("2ms"));



  NetDeviceContainer devices;
  // The install method installs devices on the nodes.
  // Creates a PointToPointNetDevice on each Node in the NodeContainer.
  devices = pointToPoint.Install (nodes);

  // Install the protocol stack on each node in the NodeContainer.
  InternetStackHelper stack;
  stack.Install (nodes);
```

```
  // Declare an address helper object.
Ipv4AddressHelper address;
// Assign addresses starting from 10.1.1.0 with subnet mask 255.255.255.0.
address.SetBase ("10.1.1.0", "255.255.255.0");

  // In ns-3, interfaces are used to associate devices with IP addresses.
Ipv4InterfaceContainer interfaces = address.Assign (devices);



  // Install applications on the application layer.
UdpEchoServerHelper echoServer (9); // Server side

ApplicationContainer serverApps = echoServer.Install (nodes.Get (1));
serverApps.Start (Seconds (1.0)); // The echoServer starts at 1 second.
serverApps.Stop (Seconds (10.0)); // The echoServer stops at 10 seconds (optional).



  // Initialize the helper object using the address and port number (9) of Node 1.
UdpEchoClientHelper echoClient (interfaces.GetAddress (1), 9);
// The maximum number of packets that can be sent.
echoClient.SetAttribute ("MaxPackets", UintegerValue (1));
// The time to wait between two packets.
echoClient.SetAttribute ("Interval", TimeValue (Seconds (1.0)));
// The size of each packet.
echoClient.SetAttribute ("PacketSize", UintegerValue (1024));

ApplicationContainer clientApps = echoClient.Install (nodes.Get (0));
clientApps.Start (Seconds (2.0)); // The client starts at 2 seconds.
clientApps.Stop (Seconds (10.0)); // The client stops at 10 seconds.



Simulator::Run (); // Start the simulator.
```

Make some changes:

1. Increase the number of packets sent by the client.
2. Change the interval between packets.
3. Change the channel attributes (datarate, delay, ……).
4. Add a second client to send packets to the server.

https://www.youtube.com/watch?v=4yg8gMIAXZA&list=PLkruFy_9uWi1v3kvhQQ90ad6IFGHDuCeR&index=6&ab_channel=AdilAlsuhaim

second.cc

The network topology consists of a mix of Point-to-Point (P2P) and CSMA (Carrier Sense Multiple Access) network configurations. Here's a detailed breakdown:

```
    10.1.1.0
n0 -------------- n1   n2   n3   n4
  point-to-point  |    |    |    |
                  ================
                    LAN 10.1.2.0
```

Components of the Network:

1. **Point-to-Point Link:**
   - **Nodes:** `n0` and `n1`
   - **IP Range:** `10.1.1.0`
   - **Description:** A direct connection between two nodes (`n0` and `n1`) with specified data rate and delay. This simulates a dedicated link such as a direct wired connection or a point-to-point wireless link.
2. **CSMA LAN:**
   - **Nodes:** `n1`, `n2`, `n3`, `n4`
   - **IP Range:** `10.1.2.0`
   - **Description:** A local area network (LAN) where multiple nodes (`n1` to `n4`) are connected using the CSMA protocol. CSMA is used to manage how data is transmitted over the shared medium to avoid collisions.

```
#include "ns3/core-module.h"
#include "ns3/network-module.h"
#include "ns3/csma-module.h"
#include "ns3/internet-module.h"
#include "ns3/point-to-point-module.h"
#include "ns3/applications-module.h"
#include "ns3/ipv4-global-routing-helper.h"


using namespace ns3;

NS_LOG_COMPONENT_DEFINE ("SecondScriptExample");

int
main (int argc, char *argv[])
{
  bool verbose = true;
```

```
uint32_t nCsma = 3;

CommandLine cmd (__FILE__);
cmd.AddValue ("nCsma", "Number of \"extra\" CSMA nodes/devices", nCsma);
cmd.AddValue ("verbose", "Tell echo applications to log if true", verbose);

cmd.Parse (argc,argv);

if (verbose)
  {
    LogComponentEnable ("UdpEchoClientApplication", LOG_LEVEL_INFO);
    LogComponentEnable ("UdpEchoServerApplication", LOG_LEVEL_INFO);
  }

nCsma = nCsma == 0 ? 1 : nCsma;

NodeContainer p2pNodes;
p2pNodes.Create (2);

NodeContainer csmaNodes;
csmaNodes.Add (p2pNodes.Get (1));
csmaNodes.Create (nCsma);

PointToPointHelper pointToPoint;
pointToPoint.SetDeviceAttribute ("DataRate", StringValue ("5Mbps"));
pointToPoint.SetChannelAttribute ("Delay", StringValue ("2ms"));

NetDeviceContainer p2pDevices;
p2pDevices = pointToPoint.Install (p2pNodes);

CsmaHelper csma;
csma.SetChannelAttribute ("DataRate", StringValue ("100Mbps"));
csma.SetChannelAttribute ("Delay", TimeValue (NanoSeconds (6560)));

NetDeviceContainer csmaDevices;
csmaDevices = csma.Install (csmaNodes);

InternetStackHelper stack;
stack.Install (p2pNodes.Get (0));
stack.Install (csmaNodes);

Ipv4AddressHelper address;
address.SetBase ("10.1.1.0", "255.255.255.0");
Ipv4InterfaceContainer p2pInterfaces;
p2pInterfaces = address.Assign (p2pDevices);

address.SetBase ("10.1.2.0", "255.255.255.0");
Ipv4InterfaceContainer csmaInterfaces;
csmaInterfaces = address.Assign (csmaDevices);

UdpEchoServerHelper echoServer (9);
```

```
  ApplicationContainer serverApps = echoServer.Install (csmaNodes.Get (nCsma));
  serverApps.Start (Seconds (1.0));
  serverApps.Stop (Seconds (10.0));

  UdpEchoClientHelper echoClient (csmaInterfaces.GetAddress (nCsma), 9);
  echoClient.SetAttribute ("MaxPackets", UintegerValue (1));
  echoClient.SetAttribute ("Interval", TimeValue (Seconds (1.0)));
  echoClient.SetAttribute ("PacketSize", UintegerValue (1024));

  ApplicationContainer clientApps = echoClient.Install (p2pNodes.Get (0));
  clientApps.Start (Seconds (2.0));
  clientApps.Stop (Seconds (10.0));

  Ipv4GlobalRoutingHelper::PopulateRoutingTables ();

  pointToPoint.EnablePcapAll ("second");
  csma.EnablePcap ("second", csmaDevices.Get (1), true);

  Simulator::Run ();
  Simulator::Destroy ();
  return 0;
}
```